

# FreeRTOS任务管理

## 文章目录

### 目录

[1.任务的基本概念](#)

[2.什么是多任务系统？](#)

[3.任务状态迁移](#)

[4.任务状态的概念](#)

[1.就绪（Ready）](#)

[2.运行（Running）](#)

[3.阻塞（Blocked）](#)

[4.挂起态\(Suspended\)](#)

[5.常用的任务函数讲解](#)

[1.任务挂起函数 vTaskSuspend\(\)](#)

[2.vTaskSuspendAll\(\)](#)

[3.任务恢复函数 vTaskResume\(\)](#)

[4.任务删除函数 vTaskDelete\(\)](#)

[5.任务延时函数 vTaskDelay\(\)](#)

[6.vTaskDelayUntil\(\)](#)

[6.任务管理实验](#)

[7.实验现象](#)

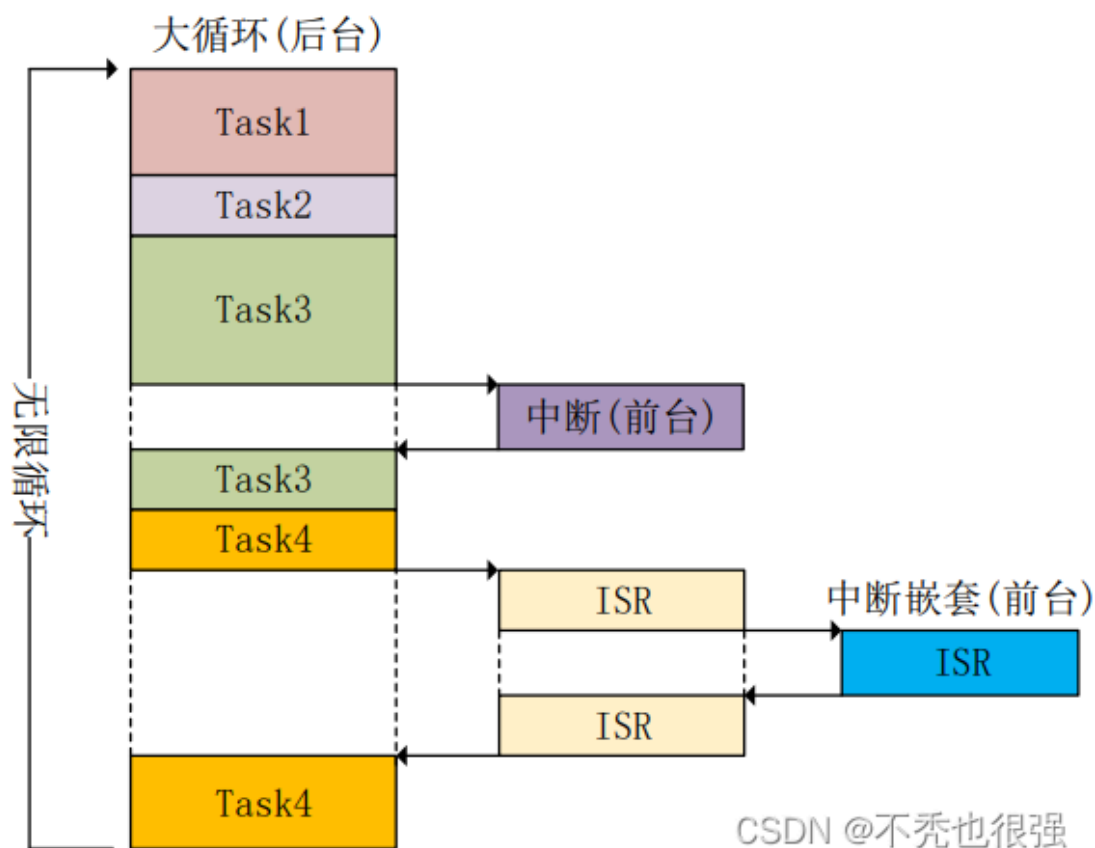
---

## 1.任务的基本概念

[RTOS](#)系统的核心就是任务管理，FreeRTOS也不例外，而且大多数学习RTOS系统的工程师或者学生主要就是为了使用RTOS的多任务处理功能，初步上手RTOS系统首先必须掌握的也是任务的创建、删除、挂起和恢复等操作，由此可见任务管理的重要性。

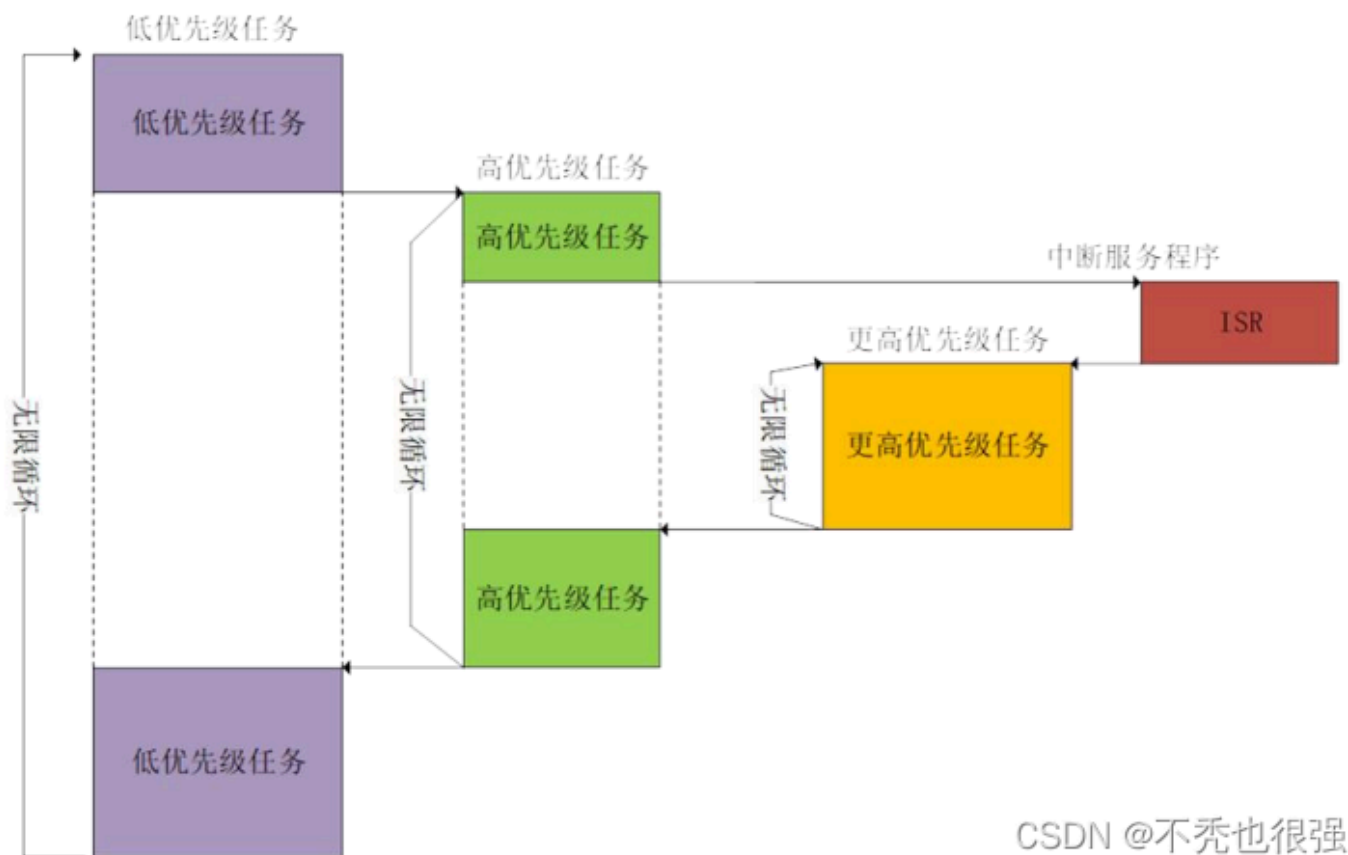
## 2.什么是多任务系统？

回想一下我们以前在使用 51、AVR、STM32 单片机裸机(未使用系统)的时候一般都是在 main 函数里面用 while(1)做一个大循环来完成所有的处理，即应用程序是一个无限的循环，循环中调用相应的函数完成所需的处理。有时候我们也需要中断中完成一些处理。相对于多任务系统而言，这个就是单任务系统，也称作前后台系统，中断服务函数作为前台程序，大循环 while(1)作为后台程序，



CSDN @不秃也很强

前后台系统的实时性差，前后台系统各个任务(应用程序)都是排队等着轮流执行，不管你 这个程序现在有多紧急，没轮到你就只能等着！相当于所有任务(应用程序)的优先级都是一样的。但是前后台系统简单啊，资源消耗也少啊！在稍微大一点的嵌入式应用中前后台系统就明显力不从心了，此时就需要多任务系统出马了。多任务系统会把一个大问题(应用)“分而治之”，把大问题划分成很多个小问题，逐步的把小问题解决掉，大问题也就随之解决了，这些小问题可以单独的作为一个小任务来处理。这些小任务是并发处理的，注意，并不是说同一时刻一起执行很多个任务，而是由于每个任务执行的时间很短，导致看起来像是同一时刻执行了很多个任务一样。多个任务带来了一个新的问题，究竟哪个任务先运行，哪个任务后运行呢？完成这个功能的东西在 RTOS 系统中叫做任务调度器。不同的系统其任务调度器的实现方法也不同，比如 [FreeRTOS](#) 是一个抢占式的实时多任务系统，那么其任务调度器也是抢占式的，运行过程如图

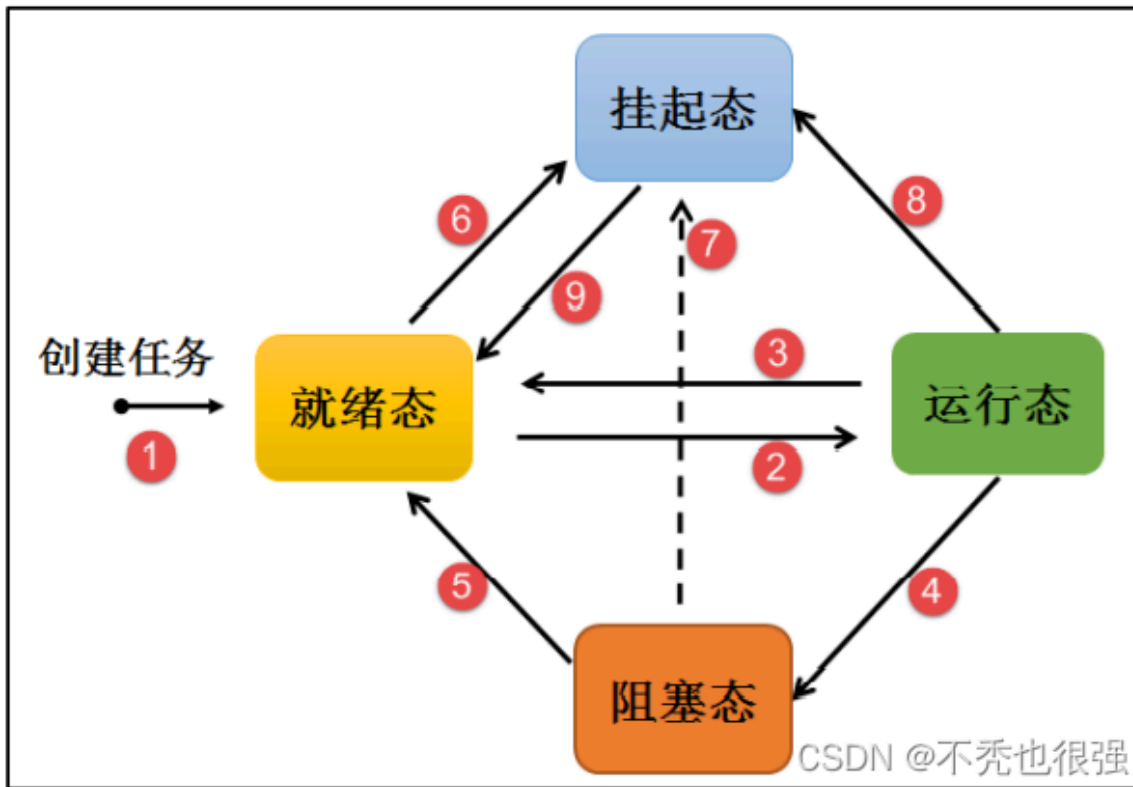


CSDN @不秃也很强

图中高优先级的任务可以打断低优先级任务的运行而取得 CPU 的使用权，这样就保证了那些紧急任务的运行。这样我们就可以为那些对实时性要求高的任务设置一个很高的优先级，比如自动驾驶中的障碍物检测任务等。高优先级的任务执行完成以后重新把 CPU 的使用权归还给低优先级的任务，这个就是抢占式多任务系统的基本原理。

### 3.任务状态迁移

FreeRTOS 系统中的每一个任务都有多种运行状态，他们之间的转换关系是怎么样的呢？从运行态任务变成阻塞态，或者从阻塞态变成就绪态，这些任务状态是如何进行迁移？下面就让我们一起了解任务状态迁移吧，



CSDN @不秃也很强

(1): 创建任务→就绪态 (Ready): 任务创建完成后进入就绪态, 表明任务已 准备就绪, 随时可以运行, 只等待调度器进行调度。

(2): 就绪态→运行态 (Running): 发生任务切换时, 就绪列表中最高优先级 的任务被执行, 从而进入运行态。

(3): 运行态→就绪态: 有更高优先级任务创建或者恢复后, 会发生任务调度, 此刻就绪列表中最高优先级任务变为运行态, 那么原先运行的任务由运行态变为就绪态, 依然在就绪列表中, 等待最高优先级的任务运行完毕继续运行原来的任务 (此处可以看做 是 CPU 使用权被更高优先级的任务抢占了)。

(4): 运行态→阻塞态 (Blocked): 正在运行的任务发生阻塞 (挂起、延时、读信号量等待) 时, 该任务会从就绪列表中删除, 任务状态由运行态变成阻塞态, 然后发生任务切换, 运行就绪列表中当前最高优先级任务。

(5): 阻塞态→就绪态: 阻塞的任务被恢复后 (任务恢复、延时时间超时、读 信号量超时或读到信号量等), 此时被恢复的任务会被加入就绪列表, 从而由阻塞态变成 就绪态; 如果此时被恢复任务的优先级高于正在运行任务的优先级, 则会发生任务切换, 将该任务将再次转换任务状态, 由就绪态变成运行态。

(6) (7) (8): 就绪态、阻塞态、运行态→挂起态 (Suspended): 任务可以通过调用 `vTaskSuspend()` API 函数都可以将处于任何状态的任务挂起, 被挂起的任务得不到 CPU 的使用权, 也不会参与调度, 除非它从挂起态中解除。

(9): 挂起态→就绪态: 把一个挂起状态 的任务恢复的唯一途径就是调用 `vTaskResume()` 或 `vTaskResumeFromISR()` API 函数, 如果此时被恢复任务的优先级高于正在运行任务的优先级, 则会发生任务切换, 将该任务将再次转换任务状态, 由就绪态 变成运行态。

## 4.任务状态的概念

FreeRTOS 系统中的每一任务都有多种运行状态。系统初始化完成后，创建的任务就可以在系统中竞争一定的资源，由内核进行调度。

任务状态通常分为以下四种：

### 1.就绪（Ready）

该任务在就绪列表中，就绪的任务已经具备执行的能力，只等待调度器进行调度，新创建的任务会初始化为就绪态。

### 2. 运行（Running）

该状态表明任务正在执行，此时它占用处理器，FreeRTOS 调度器选择运行的永远是处于最高优先级的就绪态任务，当任务被运行的一刻，它的任务状态就变成了运行态。

### 3.阻塞（Blocked）

如果任务当前正在等待某个时序或外部中断，我们就说这个任务处于阻塞状态，该任务不在就绪列表中。包含任务被挂起、任务被延时、任务正在等待信号量、读写队列或者等待读写事件等。

### 4.挂起态(Suspended)

处于挂起态的任务对调度器而言是不可见的，让一个任务进入挂起状态的唯一办法就是调用 `vTaskSuspend()` 函数；而把一个挂起状态的任务恢复的唯一途径就是调用 `vTaskResume()` 或 `vTaskResumeFromISR()` 函数，我们可以这么理解挂起态与阻塞态的区别，当任务有较长的时间不允许运行的时候，我们可以挂起任务，这样子调度器就不会管这个任务的任何信息，直到我们调用恢复任务的 API 函数；而任务处于阻塞态的时候，系统还需要判断阻塞态的任务是否超时，是否可以解除阻塞。

## 5.常用的任务函数讲解

### 1.任务挂起函数 `vTaskSuspend()`

挂起指定任务。被挂起的任务绝不会得到 CPU 的使用权，不管该任务具有什么优先级。任务可以通过调用 `vTaskSuspend()` 函数都可以将处于任何状态的任务挂起，被挂起的任务得不到 CPU 的使用权，也不会参与调度，它相对于调度器而言是不可见的，除非它从挂起态中解除。

### 2.`vTaskSuspendAll()`

这个函数就是比较有意思的，将所有的任务都挂起，其实源码很简单，也很有意思，不管三七二十一将调度器锁定，并且这个函数是可以进行嵌套的，说白了挂起所有任务就是挂起任务调度器。调度器被挂起后则不能进行上下文切换，但是中断还是使能的。当调度器被挂起的时候，如果有中断需要进行上下文切换，那么这个任务将会被挂起，在调度器恢复之后才执行切换任务。

### 3.任务恢复函数 `vTaskResume()`

既然有任务的挂起，那么当然一样有恢复，不然任务怎么恢复呢，任务恢复就是让挂起的任务重新进入就绪状态，恢复的任务会保留挂起前的状态信息，在恢复的时候根据挂起时的状态继续运行。如果

被恢复任务在所有就绪态任务中，处于最高优先级列表的第一位，那么系统将进行任务上下文的切换。

## 4.任务删除函数 vTaskDelete()

用于删除一个任务。当一个任务删除另外一个任务时，形参为要删除任务创建时返回的任务句柄，如果是删除自身，则形参为 NULL。要想使用该函数必须在 FreeRTOSConfig.h 中把 INCLUDE\_vTaskDelete 定义为 1，删除的任务将从所有就绪，阻塞，挂起和事件列表中删除。

vTaskDelay()在我们任务中用得非常多，每个任务都必须是死循环，并且是必须要有阻塞的情况，否则低优先级的任务就无法被运行了。要想使用 FreeRTOS 中的 vTaskDelay() 函数必须在 FreeRTOSConfig.h 中把 INCLUDE\_vTaskDelay 定义为 1 来使能。

## 6.vTaskDelayUntil()

在 FreeRTOS 中，除了相对延时函数，还有绝对延时函数 vTaskDelayUntil()，这个绝对延时常用于较精确的周期运行任务，比如我有一个任务，希望它以固定频率定期执行，而不受外部的影响，任务从上一次运行开始到下一次运行开始的时间间隔是绝对的，而不是相对的。

任务创建方法我在上一篇已经介绍过了这里不再赘述了

## 6.任务管理实验

任务管理实验是将任务常用的函数进行一次实验，在野火 STM32 开发板上进行该试验，通过创建两个任务，一个是 LED 任务，另一个是按键任务，LED 任务是显示任务运行的状态，而按键任务是通过检测按键的按下与否来进行对 LED 任务的挂起与恢复

代码

```
1. /* FreeRTOS头文件 */
2. #include "FreeRTOS.h"
3. #include "task.h"
4. /* 开发板硬件bsp头文件 */
5. #include "bsp_led.h"
6. #include "bsp_usart.h"
7. #include "bsp_key.h"
8. /***** 任务句柄 *****/
9. /*
10. * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
11. * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
12. * 这个句柄可以为NULL。
13. */
14. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
15. static TaskHandle_t LED_Task_Handle = NULL; /* LED任务句柄 */
16. static TaskHandle_t KEY_Task_Handle = NULL; /* KEY任务句柄 */
17.
18. /***** 内核对象句柄 *****/
19. /*
20. * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
21. * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
22. * 们就可以通过这个句柄操作这些内核对象。
23. *
24. * 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
25. * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
26. * 来完成的
27. *
28. */
29.
30.
```

```

31. /***** 全局变量声明 *****/
32. /*
33.  * 当我们在写应用程序的时候，可能需要用到一些全局变量。
34.  */
35.
36.
37. /*
38. *****/
39.  * 函数声明
40. *****/
41. */
42. static void AppTaskCreate(void);/* 用于创建任务 */
43.
44. static void LED_Task(void* pvParameters);/* LED_Task任务实现 */
45. static void KEY_Task(void* pvParameters);/* KEY_Task任务实现 */
46.
47. static void BSP_Init(void);/* 用于初始化板载相关资源 */
48.
49. /*****
50.  * @brief 主函数
51.  * @param 无
52.  * @retval 无
53.  * @note 第一步：开发板硬件初始化
54.          第二步：创建APP应用任务
55.          第三步：启动FreeRTOS，开始多任务调度
56.  *****/
57. int main(void)
58. {
59.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
60.
61.     /* 开发板硬件初始化 */
62.     BSP_Init();
63.
64.     printf("这是一个FreeRTOS任务管理实验！\n\n");
65.     printf("按下KEY1挂起任务，按下KEY2恢复任务\n");
66.
67.     /* 创建AppTaskCreate任务 */
68.     xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate, /* 任务入口函数 */
69.                          (const char* )"AppTaskCreate",/* 任务名字 */
70.                          (uint16_t )512, /* 任务栈大小 */
71.                          (void* )NULL,/* 任务入口函数参数 */
72.                          (UBaseType_t )1, /* 任务的优先级 */
73.                          (TaskHandle_t* )&AppTaskCreate_Handle);/* 任务控制块指针 */
74.     /* 启动任务调度 */
75.     if(pdPASS == xReturn)
76.         vTaskStartScheduler(); /* 启动任务，开启调度 */
77.     else
78.         return -1;
79.
80.     while(1); /* 正常不会执行到这里 */
81. }
82.
83.
84. /*****
85.  * @ 函数名 : AppTaskCreate
86.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
87.  * @ 参数 : 无
88.  * @ 返回值 : 无
89.  *****/
90. static void AppTaskCreate(void)
91. {
92.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
93.
94.     taskENTER_CRITICAL(); //进入临界区
95.
96.     /* 创建LED_Task任务 */
97.     xReturn = xTaskCreate((TaskFunction_t )LED_Task, /* 任务入口函数 */
98.                          (const char* )"LED_Task",/* 任务名字 */
99.                          (uint16_t )512, /* 任务栈大小 */
100.                          (void* )NULL, /* 任务入口函数参数 */
101.                          (UBaseType_t )2, /* 任务的优先级 */
102.                          (TaskHandle_t* )&LED_Task_Handle);/* 任务控制块指针 */

```

```

103.     if(pdPASS == xReturn)
104.         printf("创建LED_Task任务成功!\r\n");
105.     /* 创建KEY_Task任务 */
106.     xReturn = xTaskCreate((TaskFunction_t )KEY_Task, /* 任务入口函数 */
107.                          (const char*      )"KEY_Task", /* 任务名字 */
108.                          (uint16_t         )512, /* 任务栈大小 */
109.                          (void*            )NULL, /* 任务入口函数参数 */
110.                          (UBaseType_t      )3, /* 任务的优先级 */
111.                          (TaskHandle_t*    )&KEY_Task_Handle); /* 任务控制块指针 */
112.     if(pdPASS == xReturn)
113.         printf("创建KEY_Task任务成功!\r\n");
114.
115.     vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
116.
117.     taskEXIT_CRITICAL(); //退出临界区
118. }
119.
120.
121.
122. /*****
123.  * @ 函数名 : LED_Task
124.  * @ 功能说明: LED_Task任务主体
125.  * @ 参数 :
126.  * @ 返回值 : 无
127.  *****/
128. static void LED_Task(void* parameter)
129. {
130.     while (1)
131.     {
132.         LED1_ON;
133.         printf("LED_Task Running, LED1_ON\r\n");
134.         vTaskDelay(500); /* 延时500个tick */
135.
136.         LED1_OFF;
137.         printf("LED_Task Running, LED1_OFF\r\n");
138.         vTaskDelay(500); /* 延时500个tick */
139.     }
140. }
141.
142. /*****
143.  * @ 函数名 : LED_Task
144.  * @ 功能说明: LED_Task任务主体
145.  * @ 参数 :
146.  * @ 返回值 : 无
147.  *****/
148. static void KEY_Task(void* parameter)
149. {
150.     while (1)
151.     {
152.         if( Key_Scan(KEY1_GPIO_PORT, KEY1_GPIO_PIN) == KEY_ON )
153.             /* K1 被按下 */
154.             printf("挂起LED任务! \n");
155.             vTaskSuspend(LED_Task_Handle); /* 挂起LED任务 */
156.             printf("挂起LED任务成功! \n");
157.         }
158.         if( Key_Scan(KEY2_GPIO_PORT, KEY2_GPIO_PIN) == KEY_ON )
159.             /* K2 被按下 */
160.             printf("恢复LED任务! \n");
161.             vTaskResume(LED_Task_Handle); /* 恢复LED任务! */
162.             printf("恢复LED任务成功! \n");
163.         }
164.         vTaskDelay(20); /* 延时20个tick */
165.     }
166. }
167.
168. /*****
169.  * @ 函数名 : BSP_Init
170.  * @ 功能说明: 板级外设初始化, 所有板子上的初始化均可放在这个函数里面
171.  * @ 参数 :
172.  * @ 返回值 : 无
173.  *****/
174. static void BSP_Init(void)

```



```

175. {
176.     /*
177.     * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
178.     * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，
179.     * 都统一用这个优先级分组，千万不要再分组，切忌。
180.     */
181.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
182.
183.     /* LED 初始化 */
184.     LED_GPIO_Config();
185.
186.     /* 串口初始化 */
187.     USART_Config();
188.
189.     /* 按键初始化 */
190.     Key_GPIO_Config();
191.
192. }

```

## 7.实验现象

这是一个FreeRTOS任务管理实验！

按下KEY1挂起任务，按下KEY2恢复任务

创建LED\_Task任务成功!

创建KEY\_Task任务成功!

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

挂起LED任务!

挂起LED任务成功!

挂起LED任务!

挂起LED任务成功!

恢复LED任务!

恢复LED任务成功!

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

恢复LED任务!

恢复LED任务成功!

LED\_Task Running,LED1\_ON

LED\_Task Running,LED1\_OFF

LED\_Task Running,LED1\_ON

挂起LED任务!

挂起LED任务成功!

CSDN @不秃也很强

